Unit I

Enumerations, Autoboxing, and Annotations (Metadata)

Enumerations:

Before **JDK 5**, Java did not support enumerations. An **enumeration** is a list of named constants. Although final variables were used earlier, they were not a complete replacement.

From **JDK 5**, Java introduced enumerations. Unlike some languages where enums are just integer constants, **Java enums are class types**. Therefore, they can have **constructors, methods, and instance variables**, making them more powerful.

Enumeration fundamentals:

An **enumeration** is created using the enum keyword and defines a list of named constants. Example:

enum Apple { Jonathan, GoldenDel, RedDel, Winesap, Cortland }

The identifiers inside the enum are called **enumeration constants**. Each constant is automatically **public, static, and final**, and its type is the enum itself (Apple). Such constants are called **self-typed**.

After defining an enum, a variable of that type can be declared. Even though enums are class types, they are **not instantiated using new**.

Apple ap;

ap = Apple.RedDel;

An enum variable can hold **only the values defined in that enum**. Enum constants can be compared using the == operator.

if (ap == Apple.GoldenDel)

Enums can also be used in **switch statements**. In case labels, constants are written **without the enum name**.

```
switch(ap) {
   case Jonathan:
                // . . .
   case Winesap:
                // . . .
}
```

When an enum constant is printed, **its name is displayed**.

System.out.println(Apple.Winesap); // Outputs: Winesap

```
// An enumeration of apple varieties.
enum Apple {
  Jonathan, GoldenDel, RedDel, Winesap, Cortland
}

class EnumDemo {
  public static void main(String args[])
  {
    Apple ap;

    ap = Apple.RedDel;

    // Output an enum value.
    System.out.println("Value of ap: " + ap);
    System.out.println();

    ap = Apple.GoldenDel;

    // Compare two enum values.
    if(ap == Apple.GoldenDel)
      System.out.println("ap contains GoldenDel.\n");

    // Use an enum to control a switch statement.
    switch(ap) {
      case Jonathan:
        System.out.println("Jonathan is red.");
        break;
      case GoldenDel:
        System.out.println("Golden Delicious is yellow.");
        break;
      case RedDel:
        System.out.println("Red Delicious is red.");
        break;
      case Winesap:
        System.out.println("Winesap is red.");
        break;
      case Cortland:
        System.out.println("Cortland is red.");
        break;
    }
  }
}
```

The output from the program is shown here:

```
Value of ap: RedDel

ap contains GoldenDel.

Golden Delicious is yellow.
```

The values() and valueOf() Methods:

All Java enumerations automatically contain two predefined methods: **values()** and **valueOf()**.

- values() returns an array containing all the enumeration constants.
- valueOf() returns the enumeration constant whose name matches the given string.

Example:

```
enum Apple {
   Jonathan, GoldenDel, RedDel, Winesap, Cortland
}

class EnumDemo {
   public static void main(String args[]) {

      // Using values()
      Apple allApples[] = Apple.values();
      for (Apple a : allApples)
         System.out.println(a);

      // Using valueOf()
      Apple ap = Apple.valueOf("Winesap");
      System.out.println("ap contains " + ap);
   }
}
```

Output

```
Jonathan
GoldenDel
RedDel
Winesap
Cortland
ap contains Winesap
```

Example using values() to iterate through enum constants:

```
for (Apple a : Apple.values())
{
   System.out.println(a);
}
```

Differentiate values() and valueOf() methods:

values():

- It returns an array of all enum constants
- Used to iterate through enum values
- Does not take any parameter

valueOf():

- It returns one enum constant
- Used to get a constant by name
- Takes a String parameter

Java enumerations are class types

In Java, an **enumeration is a class type**. Although enum objects are not created using new, they have many features of normal classes. An enum can have **constructors, instance variables, methods**, and can **implement interfaces**.

Each **enumeration constant is an object** of the enum type. When an enum has a constructor, it is **called once for each constant** at the time the enum is created. Each constant also has its **own copy of instance variables**.

Example:

```
enum Apple
{
   Jonathan(10), GoldenDel(9), RedDel(12);
   private int price;
   Apple(int p)
   {
      price = p;
   }
   int getPrice()
   {
      return price;
   }
}

class EnumClassDemo
{
   public static void main(String args[])
   {
      System.out.println("Price of RedDel: " + Apple.RedDel.getPrice());
   }
}
```

In the Apple enum, price stores the cost of each apple. The constructor sets the price, and getPrice() returns it. Each enum constant has its own price, so the values are independent.

Enums can have **overloaded constructors**. If a constant does not pass a value, a **default constructor** is used.

Restrictions of Enums:

- An enum **cannot inherit another class**
- An enum **cannot be extended** (cannot be a superclass)

Enumerations Inherit Enum

In Java, an enumeration cannot extend another class, but **all enums automatically inherit java.lang.Enum** class. This class provides useful methods such as ordinal(), compareTo(), and equals(). Each enumeration constant is an **object** of the enum type and inherits these methods from the Enum class.

Important Enum Methods:

- **ordinal()**
  The ordinal() method returns the **position of an enumeration constant** in the enum list. The counting starts from **0**.
  Syntax: final int ordinal()
  Example: Apple.Jonathan.ordinal(); // returns 0
  Explanation: Apple is an enumeration. Enumeration indexing starts from zero, and Jonathan is the first constant in the Apple enum, so ordinal() returns 0.
- **compareTo()**
  Compares two enum constants of the same enum based on their ordinal values. Returns:
  - negative value if first comes before second
  - zero if both are equal
  - positive value if first comes after second
- **equals()**
  Checks whether two references point to the **same enum constant** in the same enum. Returns true only if both refer to the same constant.

Enum constants can also be compared using the == operator.

Example:

```
enum Apple {
   Jonathan, GoldenDel, RedDel
}
class EnumDemo {
   public static void main(String args[]) {
      Apple a1 = Apple.Jonathan;
      Apple a2 = Apple.RedDel;
      System.out.println(a1.ordinal());
      System.out.println(a1.compareTo(a2));
      System.out.println(a1.equals(a2));
   }
}
```

Explanation:

The enum Apple automatically inherits the Enum class. Hence, its constants can directly use methods like ordinal(), compareTo(), and equals() without defining them.

Type wrappers:

Type wrappers are used in Java to **represent primitive data types as objects**. They are required when:

- An object representation is needed (for example, in collections)
- A primitive value must be passed by reference Wrapper classes allow primitive types to be integrated into Java's **object-oriented framework**.

The wrapper classes are: Double, Float, Long, Integer, Short, Byte, Character, Boolean

---

Character

Character is a wrapper around a char.

Constructor: Character(char ch)

Method: charValue() returns the encapsulated character.

Boolean

Boolean is a wrapper around boolean values.

Constructors:

- Boolean(boolean boolValue)
- Boolean(String boolString)

If the string is "true" (uppercase or lowercase), the object is true; otherwise false.

Method: booleanValue() returns the boolean value.

Byte

Byte is a numeric type wrapper used to represent numeric values.

It inherits methods from the Number class to return values in different numeric formats.

Short

Short is a numeric type wrapper that represents numeric values.

It inherits methods from Number to return the value in different number formats.

Integer

Integer is a numeric wrapper class.

Constructors:

- Integer(int num)
- Integer(String str)

If the string does not contain a valid number, a NumberFormatException is thrown.

It implements methods from Number such as intValue().

Long

Long is a numeric wrapper class used to represent numeric values.

It inherits conversion methods from the Number class.

Float

Float is a numeric wrapper class that represents floating-point values.
It inherits methods from Number to return values in different numeric formats.

Double

Double is a numeric wrapper class used to represent double-precision values.
Method: doubleValue() returns the value as a double.

Numeric Wrapper Common Methods (from Number)

All of the numeric type wrappers inherit the abstract class Number. Number declares methods that return the value of an object in each of the different number formats. These methods are shown here:

```
byte byteValue( )
double doubleValue( )
float floatValue( )
int intValue( )
long longValue( )
short shortValue( )
```

For example, doubleValue( ) returns the value of an object as a double, floatValue( ) returns the value as a float, and so on. These methods are implemented by each of the numeric type wrappers.

Example:

```
// Demonstrate a type wrapper.
class Wrap {
   public static void main(String args[]) {
      Integer iOb = new Integer(100);   // Boxing
      int i = iOb.intValue();           // Unboxing
      System.out.println(i + " " + iOb); // displays 100 100
   }
}
```

- All wrapper classes override the toString() method.
- toString() returns the human-readable value stored in the wrapper.
- Wrapper objects can be printed directly using println().
- **Boxing** means wrapping a primitive value inside an object.
- **Unboxing** means extracting the primitive value from a wrapper object.

This program wraps the integer value 100 inside an Integer object called iOb. The program then obtains this value by calling intValue( ) and stores the result in i.

The process of encapsulating a value within an object is called boxing. Thus, in the program, this line boxes the value 100 into an Integer:

```
Integer iOb = new Integer(100);
```

The process of extracting a value from a type wrapper is called unboxing. For example, the program unboxes the value in iOb with this statement:

```
int i = iOb.intValue();
```

Autoboxing:

From **JDK 5**, Java introduced **autoboxing** and **auto-unboxing**.

Autoboxing automatically converts a primitive type into its corresponding wrapper object when needed. here is no need to explicitly create the wrapper object using new.

Auto-unboxing automatically converts a wrapper object into its primitive type when required. There is no need to call methods such as intValue() or doubleValue().

Example:

```
class AutoBox
{
   public static void main(String args[])
  {
      Integer iOb = 100; // autoboxing
      int i = iOb;       // auto-unboxing
      System.out.println(i + " " + iOb);
   }
}
```

This makes Java code **simpler and cleaner** by avoiding manual boxing and unboxing.
They help **prevent errors** and are important for **generics**, which operate only on objects. Autoboxing also makes working with the **Collections Framework** easier.

Autoboxing and Methods:

Autoboxing and auto-unboxing also occur when arguments are passed to methods and when values are returned from methods.

If a method expects a wrapper object and a primitive value is passed, the value is autoboxed. If a method returns a primitive value and it is assigned to a wrapper variable, the value is autoboxed again.

Inside the method, a wrapper object is auto-unboxed when its primitive value is needed.

Example:

```
// Autoboxing/unboxing with method parameters and return values
class AutoBox2 {
   static int m(Integer v) {
      return v; // auto-unboxing
   }

   public static void main(String args[]) {
      Integer iOb = m(100); // autoboxing
      System.out.println(iOb);
   }
}
```

Explanation: The value 100 is autoboxed when passed to the method, auto-unboxed inside the method, and autoboxed again when the return value is assigned.

Autoboxing/Unboxing occurs in Expressions:

Autoboxing and auto-unboxing occur **whenever a conversion between a primitive type and a wrapper object is required**, including inside **expressions**.

Within an expression, wrapper objects are **automatically unboxed**, the expression is evaluated, and the result is **reboxed if required**.

Example: Autoboxing in Expressions

```
// Autoboxing/unboxing occurs inside expressions.
class AutoBox3 {
   public static void main(String args[]) {
      Integer iOb, iOb2;
      int i;
      iOb = 100;
      System.out.println("Original value of iOb: " + iOb);

      ++iOb;                          // unbox, increment, rebox
      System.out.println("After ++iOb: " + iOb);

      iOb2 = iOb + (iOb / 3);         // unbox, evaluate, rebox
      System.out.println("iOb2 after expression: " + iOb2);

      i = iOb + (iOb / 3);            // unbox, evaluate
      System.out.println("i after expression: " + i);
   }
}
```

In the statement ++iOb, the value is **unboxed**, incremented, and then **reboxed**.

Autoboxing/Unboxing Boolean and character values:

Java provides wrapper classes **Boolean** and **Character** for the primitive types boolean and char.

Autoboxing and auto-unboxing apply to these wrappers as well.

Example: // Autoboxing/unboxing a Boolean and Character.

```
class AutoBox5
{
   public static void main(String args[])
   {

      Boolean b = true;   // autoboxing
      if(b)
        System.out.println("b is true");      // auto-unboxing
      Character ch = 'x';                 // autoboxing
      char ch2 = ch;                      // auto-unboxing
      System.out.println("ch2 is " + ch2);
   }
}
```

Explanation:

- The Boolean object b is **auto-unboxed** when used in the if condition.
- A Boolean object can control if, while, for, and do-while statements.
- The Character object is **auto-unboxed** when assigned to a char.

This makes using Boolean and Character objects **simple and natural** without manual conversion.

Autoboxing/Unboxing helps prevent errors:

Autoboxing and auto-unboxing help **prevent errors** that can occur during manual boxing and unboxing.

Example: // An error produced by manual unboxing.

```
class UnboxingError {
   public static void main(String args[]) {
      Integer iOb = 1000;       // autoboxing
      int i = iOb.byteValue(); // manual unboxing
      System.out.println(i);
   }
}
```

In this program, manually unboxing the value using byteValue() causes **truncation**, and the output is **–24 instead of 1000**. This happens because the value is converted to an incompatible type.

Auto-unboxing prevents such errors because it always unboxes the value into a **compatible primitive type**. Autoboxing and auto-unboxing ensure correct object and value conversions, making the code **safer and more reliable**.

A word of Warning.

Although Java supports autoboxing and auto-unboxing, **wrapper objects should not be used in place of primitive types in all cases**.

Using wrapper classes like Double instead of primitive double for calculations is a **bad use of autoboxing/unboxing** because it adds unnecessary overhead and is **less efficient**.

Example: // A bad use of autoboxing/unboxing!

```
Double a, b, c;
a = 10.0;
b = 4.0;
c = Math.sqrt(a*a + b*b);
System.out.println("Hypotenuse is " + c);
```

Although this code works correctly, it is inefficient because **each autobox and auto-unbox adds overhead**.

In general, **type wrappers should be used only when an object representation is required**. Autoboxing/unboxing was not meant to replace primitive types in Java.

Annotations (Metadata)

Annotations are a feature introduced in **JDK 5.**

Annotations allow **supplemental information** to be embedded into a Java source file. This information **does not change the behavior or semantics** of a program, but is used by **tools during development and deployment**, such as source-code generators. Annotations are also known as **metadata.**

Annotation basics

An annotation is created using a mechanism **based on interfaces**. An annotation type is declared using @interface.

Example: Defining an Annotation

```
@interface MyAnno
{
   String str();
   int val();
}
```

- The @ symbol before interface indicates that an **annotation type** is being declared.
- Annotation members are **method declarations without bodies**.
- These methods act like **fields** when values are assigned.

An annotation **cannot extend another interface**, but all annotations automatically extend the **Annotation interface**, which is defined in the java.lang.annotation package.

Using an Annotation

Once an annotation is declared, it can be applied to **any declaration**, such as: classes, methods, fields, parameters, enum constants.

The annotation **always appears before** the declaration.

Example: Applying an Annotation

```
@MyAnno(str = "Annotation Example", val = 100)
public static void myMeth()
{
   // ...
}
```

- The annotation name is preceded by @.
- Member values are assigned using name = value.
- Annotation members are used **without parentheses**, like fields.

This annotation is associated with the method myMeth().

Specifying retention policy

An **annotation retention policy** determines **how long an annotation is retained**.

How to set retention policy?

A retention policy is specified using the built-in annotation @Retention.

Syntax: @Retention(retention-policy)

Example:

```
@Retention(RetentionPolicy.RUNTIME)
@interface MyAnno
{
   String str();
   int val();
}
```

This makes the annotation available during program execution.

If no retention policy is specified, the **default is CLASS**.

Java defines **three retention policies** in the RetentionPolicy enumeration: (Types)

- **SOURCE :** The annotation is retained only in the source file and discarded during compilation.
- **CLASS :** The annotation is stored in the .class file but is **not available at run time**.
- **RUNTIME :** The annotation is stored in the .class file and is **available at run time through the JVM**. This provides the **highest level of retention**.

Obtaining Annotations at runtime by use of reflection

If an annotation has **RUNTIME retention**, it can be accessed at run time using **reflection**. Reflection allows a program to obtain information about a class during execution.

- The first step is to obtain a **Class object** that represents the class. One way to do this is by calling getClass().
  Syntax: final Class getClass()
- After obtaining the Class object, methods such as getMethod() can be used to get information about methods.
  Syntax: Method getMethod(String methName, Class ... paramTypes)
- To obtain an annotation associated with a class element, the method getAnnotation() is used.
  Syntax: Annotation getAnnotation(Class annoType)

This method returns the annotation if it exists, or null if it is not found or does not have RUNTIME retention.

Reflection Example (Method Without Parameters)

```
import java.lang.annotation.*;
import java.lang.reflect.*;
@Retention(RetentionPolicy.RUNTIME)
@interface MyAnno
{
   String value();
}

class Demo
{
   @MyAnno("Hello Annotation")
   public static void show() throws Exception
   {
      Method m = Demo.class.getMethod("show");
      MyAnno a = m.getAnnotation(MyAnno.class);

      System.out.println(a.value());
   }
   public static void main(String[] args) throws Exception
   {
      show();
   }
}
```

Output:

Hello Annotation

- MyAnno.class is a **class literal**, which provides the Class object for the annotation.
- Annotation member values are accessed using **method-call syntax**.

Reflection Example (Method With Parameters)

When a method has parameters, their **Class objects must be specified** in getMethod().

```
import java.lang.annotation.*;
import java.lang.reflect.*;
@Retention(RetentionPolicy.RUNTIME)
@interface MyAnno
{
   String value();
}
class Demo
{
   @MyAnno("Two Parameters")
   public static void show(String s, int i) throws Exception
   {
      Method m = Demo.class.getMethod("show", String.class, int.class);
      MyAnno a = m.getAnnotation(MyAnno.class);
      System.out.println(a.value());
   }
   public static void main(String[] args) throws Exception
   {
      show("test", 10);
   }
}
```

Output : Two Parameters

Here, String.class and int.class specify the parameter types of the method.

Obtaining All Annotations

All annotations with **RUNTIME retention** associated with a class, method, constructor, or field can be obtained using the getAnnotations() method.

getAnnotations() returns **all runtime annotations** associated with a class or method as an array of Annotation objects.

Syntax:

Annotation[] getAnnotations()

- It returns an **array of Annotation objects**
- It can be called on Class, Method, Constructor, and Field
- Each annotation is displayed using its overridden toString() method

Example:

```
import java.lang.annotation.*;
import java.lang.reflect.*;
@Retention(RetentionPolicy.RUNTIME)
@interface Info
{
   String value();
}
@Info("Class Annotation")
class Demo
{
   @Info("Method Annotation")
   public static void show() throws Exception
   {
      // Get all annotations of class
      Annotation[] classAnnos = Demo.class.getAnnotations();
      for (Annotation a : classAnnos)
         System.out.println(a);
      // Get all annotations of method
      Method m = Demo.class.getMethod("show");
      Annotation[] methodAnnos = m.getAnnotations();
      for (Annotation a : methodAnnos)
         System.out.println(a);
   }
   public static void main(String[] args) throws Exception
   {
      show();
   }
}
```

Output:

```
@Info(value=Class Annotation)
@Info(value=Method Annotation)
```

The AnnotatedElement Interface

The methods getAnnotation() and getAnnotations() are defined by the **AnnotatedElement interface**, which is part of the java.lang.reflect package.

This interface provides **reflection support for annotations** and is implemented by the following classes: Method, Field, Constructor, Class and Package.

AnnotatedElement is the base interface that allows annotations to be accessed using reflection.

Methods of AnnotatedElement

- **getDeclaredAnnotations()**
  Annotation[] getDeclaredAnnotations()
  Returns **all non-inherited annotations** present in the invoking object.
- **isAnnotationPresent()**
  boolean isAnnotationPresent(Class annoType)
  - Returns true if the specified annotation is present.
  - Returns false if the annotation is not present.

Using Default values

Annotation members can be given **default values**, which are used when no value is specified while applying the annotation.

A default value is assigned using the default keyword in the member declaration.

General Form:

type member() default value;

The default value must be compatible with the member's type.

Example: Annotation with Default Values

```
@Retention(RetentionPolicy.RUNTIME)
@interface MyAnno
{
   String str() default "Testing";
   int val() default 9000;
}
```

Here:

- str has the default value "Testing"
- val has the default value 9000

Ways to Use the Annotation :

```
@MyAnno()                    // both values use defaults
@MyAnno(str = "some string")    // val uses default
@MyAnno(val = 100)             // str uses default
@MyAnno(str = "Testing", val = 100) // no defaults used
```

Example Program Using Default Values

```
import java.lang.annotation.*;
import java.lang.reflect.*;
@Retention(RetentionPolicy.RUNTIME)
@interface MyAnno
{
   String str() default "Testing";
   int val() default 9000;
}
```

```
class Meta3
{
   @MyAnno()
   public static void myMeth()
   {
      try {
         Method m = Meta3.class.getMethod("myMeth");
         MyAnno anno = m.getAnnotation(MyAnno.class);
         System.out.println(anno.str() + " " + anno.val());
      } catch (NoSuchMethodException e) {
         System.out.println("Method Not Found.");
      }
   }

   public static void main(String[] args)
   {
      myMeth();
   }
}
```

Output:
Testing 9000

Marker Annotations

A **marker annotation** is an annotation that **contains no members**. Its only purpose is to **mark a declaration**. The presence of the annotation itself is sufficient.

To check whether a marker annotation is present, the method isAnnotationPresent() is used. This method is defined in the **AnnotatedElement interface**.

Example of a Marker Annotation

```
import java.lang.annotation.*;
import java.lang.reflect.*;
// A marker annotation
@Retention(RetentionPolicy.RUNTIME)
@interface MyMarker { }
class Marker
{
   @MyMarker   // marker annotation
   public static void myMeth()
   {
      Marker ob = new Marker();
      try
      {
         Method m = ob.getClass().getMethod("myMeth");
         // Check if annotation is present
         if (m.isAnnotationPresent(MyMarker.class))
            System.out.println("MyMarker is present.");
      }
```

```
        catch (NoSuchMethodException e)
        {
            System.out.println("Method Not Found.");
        }
    }
    public static void main(String args[])
    {
        myMeth();
    }
}
```

Output:

MyMarker is present.

- Parentheses are **not required** when applying a marker annotation: example - @MyMarker

Single Member annotations

A **single-member annotation** is an annotation that contains **only one member**. It allows a **shorthand form** for specifying the value of that member.

To use the shorthand syntax, the **member name must be value**. In this case, the member name does not need to be specified when applying the annotation.

Example of a Single-Member Annotation

```
import java.lang.annotation.*;
import java.lang.reflect.*;
// A single-member annotation.
@Retention(RetentionPolicy.RUNTIME)
@interface MySingle
{
    int value();   // member name must be value
}
class Single
{
    @MySingle(100)
    public static void myMeth()
    {
        try
        {
            Method m = Single.class.getMethod("myMeth");
            MySingle anno = m.getAnnotation(MySingle.class);
            System.out.println(anno.value());
        }
        catch (NoSuchMethodException e)
        {
            System.out.println("Method Not Found.");
        }
```

```
    }
    public static void main(String args[])
    {
        myMeth();
    }
}
```

Output: 100

Single-Member with Default Values

If other members exist, they **must have default values** to use shorthand syntax.

```
@interface SomeAnno
{
   int value();
   int xyz() default 0;
}
Usage:
@SomeAnno(88)                // xyz defaults to 0
@SomeAnno(value = 88, xyz = 99)
```

Built-In annotations

Java provides several **built-in annotations**. Among them, **seven are general-purpose**.

Built-in Annotations from java.lang.annotation

- @Retention
- @Documented
- @Target
- @Inherited

Built-in Annotations from java.lang

- @Override
- @Deprecated
- @SuppressWarnings

@Retention

- Used **only on annotation declarations**
- Specifies **how long an annotation is retained**
- Uses values from RetentionPolicy (SOURCE, CLASS, RUNTIME)

@Documented

- A **marker annotation**
- Indicates that an annotation should be **included in documentation**
- Used only on **annotation declarations**

@Target

- Specifies **where an annotation can be applied**
- Used only on **annotation declarations**
- Takes values from the ElementType enumeration

Common ElementType values:

- ANNOTATION_TYPE – another annotation
- CONSTRUCTOR – constructor

- FIELD – field
- LOCAL_VARIABLE – local variable
- METHOD – method
- PACKAGE – package
- PARAMETER – parameter
- TYPE – class, interface, or enum

Example:

```
@Target({ElementType.FIELD, ElementType.LOCAL_VARIABLE})
```

@Inherited

- A **marker annotation**
- Used only on **annotation declarations**
- Applies only to **class-level annotations**
- Allows a subclass to **inherit an annotation from its superclass**

@Override

- A **marker annotation**
- Used only on **methods**
- Ensures the method **overrides a superclass method**
- Causes a **compile-time error** if no override occurs

@Deprecated

- A **marker annotation**
- Indicates that a declaration is **obsolete**
- Suggests that a newer alternative exists

@SuppressWarnings

- Suppresses **compiler warnings**
- Warning names are specified as **strings**
- Can be applied to **any declaration**

Built-in annotations provide **standard metadata support** for controlling annotation behavior, documentation, inheritance, and compiler checks.

Some Restrictions on Annotations

Annotations in Java are subject to the following restrictions:

- An annotation **cannot inherit another annotation**
- All methods declared in an annotation **must not have parameters**
- Annotation methods can return only:
  - A **primitive type** (such as int, double)
  - **String** or **Class**
  - An **enum type**
  - Another **annotation type**
  - An **array** of any of the above types
- Annotations **cannot be generic** and cannot take type parameters

---

Java Beans

Introduction

Java Beans are important because they allow **complex systems to be built from reusable software components**. These components can be created by the programmer or supplied by **different vendors**. Java Beans define an **architecture** that explains how these components work together.

Just like hardware components (such as resistors or integrated circuits) can be reused to build different systems, software components can also be reused. For this, a proper **component-based architecture** is required. Java Beans provide this architecture by allowing components to be selected, understood, reused, and easily updated when new versions are available.

Definition (Key characteristics)

- A **Java Bean** is a **reusable software component** designed to work in different environments. There is no restriction on what a Bean can do.
- It may perform a **simple task** (such as getting an inventory value) or a **complex task** (such as forecasting stock performance).
- A Bean can be **Visible**, such as a button in a graphical user interface or **Invisible**, such as software that decodes multimedia data
- A Bean may work **Independently** on a user's system or **Cooperatively** with other distributed components

Thus, Java Beans support **reusability, interoperability, and component-based software development**.

Advantages of java beans provide to component developers

- A Java Bean follows Java's **write-once, run-anywhere** principle.
- The **properties, events, and methods** of a Bean exposed to other applications can be controlled.
- **Auxiliary software** can be used to configure a Bean at design time and is not required at run time.
- A Bean's **configuration settings can be saved and restored** later.
- A Bean can **receive events from other objects and generate events** for other objects.

Introspection

Introspection is the process of **analyzing a Java Bean to determine its capabilities**. It allows tools (such as design tools) to find out a Bean's **properties, events, and methods**. Without introspection, **Java Beans cannot function**.

A Bean developer can expose information in **two ways**:

1. By following **naming conventions (design patterns)**
2. By providing a class that implements the **BeanInfo interface**

Design Patterns for Properties

A **property** represents part of a Bean's state.
Properties affect the **behavior and appearance** of a Bean.
Properties are accessed using:

- **Getter method** → to read a value
- **Setter method** → to modify a value

There are **two types of properties**:

- Simple properties
- Indexed properties

Simple Properties

A **simple property** has a single value. It is accessed using two components : **getter and setter methods**.

Design Pattern:

- **Getter method** – Used to **read** the property value
  public T getN()
- **Setter method** – Used to **modify** the property value
  public void setN(T value)

Together, these methods allow controlled access to a Bean's property.

- **Read/write property** → has both get and set
- **Read-only property** → has only get
- **Write-only property** → has only set

Example:

```
class BoxBean
{
   private double height;
   public double getHeight()
   {
      return height;
   }
   public void setHeight(double h)
   {
      height = h;
   }
}
```

Here, height is a simple read/write property.

Indexed Properties

An **indexed property** contains **multiple values**, like an array.

It allows access to **individual elements or the entire array**.

Design Pattern:

public T getN(int index)
public void setN(int index, T value)
public T[] getN()
public void setN(T[] values)
(where N is the name of the property and T is its type)

Example:

```
class DataBean
{
   private int data[];
   public int getData(int index)
   {
      return data[index];
   }
   public void setData(int index, int value)
   {
      data[index] = value;
   }
}
```

Design Patterns for Events

Beans use the **delegation event model**.

A Bean can:

- **Generate events**
- **Send events to other objects**

Event Listener Methods:

public void addTListener(TListener l)
public void removeTListener(TListener l)

- Multiple listeners → **multicast**
- Single listener → **unicast** (may throw TooManyListenersException)

Example

```
public void addTemperatureListener(TemperatureListener t)
{
   // add listener
}
public void removeTemperatureListener(TemperatureListener t)
{
   // remove listener
}
```

Methods and Design Patterns

- **Only public methods** are discovered by introspection
- **Private and protected methods are ignored**
- Design patterns are **not used** for non-property methods

Using the BeanInfo Interface

The **BeanInfo interface** allows a developer to **explicitly control** what information is exposed.

Important methods:

PropertyDescriptor[] getPropertyDescriptors()
EventSetDescriptor[] getEventSetDescriptors()
MethodDescriptor[] getMethodDescriptors()

These methods return information about: Properties, Events, Methods

Example:

```
import java.beans.*;
public class MyBeanBeanInfo extends SimpleBeanInfo
{
   public PropertyDescriptor[] getPropertyDescriptors()
   {
      return null;
   }
}
```

BeanInfo Naming Rule

If a Bean is named MyBean, the BeanInfo class **must be named**: MyBeanBeanInfo

SimpleBeanInfo Class

SimpleBeanInfo provides **default implementations** of BeanInfo.

- You can extend SimpleBeanInfo
- Override only required methods
- If a method is not overridden, **design-pattern introspection is used**

Bound and constraint properties

Bound Property

A **bound property** generates an event **when its value is changed**.

The event generated is of type **PropertyChangeEvent** and is sent to objects that have registered interest.

The receiving class must implement the **PropertyChangeListener** interface.

Constrained Property

A **constrained property** generates an event **when an attempt is made to change its value**. It also generates a **PropertyChangeEvent**, but registered listeners can **veto the change** by throwing a **PropertyVetoException**.

This allows a Bean to behave differently depending on its run-time environment. The receiving class must implement the **VetoableChangeListener** interface.

[**Bound property**- notifies after change; **Constrained property**- allows veto before change]

Persistence

Persistence is the ability to **save the current state of a Bean** (property values and instance variables) to **non-volatile storage** and **restore it later**. Java Beans use **object serialization** to provide persistence.

java.io.Serializable is a marker interface that enables automatic serialization of a Bean. The class must have a parameterless constructor.

The **transient** keyword is used to exclude fields from serialization, so they are not saved.

java.io.Externalizable is used when a Bean does not implement Serializable and requires explicit control over serialization.

Customizers

A **customizer** is provided by a Bean developer to **help another developer configure a Java Bean**.

It can offer **step-by-step guidance** for using the Bean in a specific context and may include **online documentation**

Java beans API

The Java Beans functionality is provided by **classes and interfaces in the java.beans package**.

These classes support **introspection, properties, events, methods, and persistence** of Java Beans.

Among all classes, the most important ones are:

- **Introspector**
- **PropertyDescriptor**
- **EventSetDescriptor**
- **MethodDescriptor**

Introspector

The **Introspector** class provides static methods that support **introspection**. Its most important method is getBeanInfo().

Method:

static BeanInfo getBeanInfo(Class<?> bean)

This method returns a **BeanInfo object** that contains information about the specified Bean.

PropertyDescriptor

The **PropertyDescriptor** class describes a **Bean property**.

It provides methods to:

- Check if a property is **bound** → isBound()
- Check if a property is **constrained** → isConstrained()
- Get the **property name** → getName()

EventSetDescriptor

The **EventSetDescriptor** class represents a **Bean event**.

It provides methods to:

- Get the method used to **add listeners** → getAddListenerMethod()
- Get the method used to **remove listeners** → getRemoveListenerMethod()
- Get the **listener type** → getListenerType()
- Get the **event name** → getName()

MethodDescriptor

The **MethodDescriptor** class represents a **Bean method**.

It provides methods to:

- Get the **method name** → getName()
- Get the **Method object** → getMethod()

Steps to be followed in creating a new Java Bean

1. **Create a public class**

   Define a public class that represents the Bean. This class acts as the reusable software component.

2. **Declare private instance variables (properties)**

   All properties of a Bean must be declared as **private** to ensure encapsulation.

3. **Provide public getter and setter methods**

   For each property, provide public **getter (get)** and **setter (set)** methods following standard naming conventions so that introspection tools can identify them.

4. **Support persistence**

   Implement the java.io.Serializable interface so that the Bean's state (property values) can be saved and restored.

5. **Add event-handling support (if required)**

   If the Bean generates or listens to events, provide appropriate methods to add and remove event listeners.

6. **Provide a BeanInfo class (optional)**

   Create a class extending SimpleBeanInfo to explicitly control which properties, events, and methods are exposed to other applications.

Example

```
import java.io.Serializable;
public class StudentBean implements Serializable
{
   // private properties
   private String name;
   private int age;

   // no-argument constructor
   public StudentBean()
   {      }
   public String getName()                  // getter method
   {
      return name;
   }
   public void setName(String n)       // setter method
   {
      name = n;
   }
   public int getAge()                      // getter method
   {
      return age;
   }
   public void setAge(int a)               // setter method
   {
      age = a;
   }
}

public class TestBean
{
   public static void main(String[] args)
   {
      StudentBean s = new StudentBean();
      s.setName("Anu");
      s.setAge(20);

      System.out.println(s.getName());
      System.out.println(s.getAge());
   }
}
```

- **Public class** → StudentBean
- **Private properties** → name, age
- **Getter and setter methods** → getName(), setName(), getAge(), setAge()
- **Persistence support** → implements Serializable
- **Reusable component** → Can be used by other applications and tools